DBMS LAB · ASSIGNMENT 8

Basic SQL Queries and Aggregate Functions

Retrieving, summarizing and changing airline reservation data

STUDENT
Reddy Sai Abhiram

GITHUB
github.com/reddysaiabhiram2468

PROJECT
Airline Reservation & Flight Booking System

DATABASE
MySQL 8.0 · `airline_reservation`

SUBMITTED
25 October 2026

Aim

To query the airline reservation database with WHERE, ORDER BY, GROUP BY and HAVING and the aggregate functions COUNT, SUM, AVG, MIN and MAX, and to change data with UPDATE and DELETE. The queries run on the Assignment 7 data, a snapshot taken at 25 October 2026, 14:00 IST. Every output is real MySQL 8.0.46 console output. The statements are saved in `sql/05_basic_queries.sql`.

Part	Topic	Queries
A	WHERE: comparison, AND/OR, BETWEEN, IN, NOT IN, LIKE	8.1 – 8.7
B	ORDER BY (ASC/DESC, two keys) and LIMIT	8.8 – 8.10
C	COUNT, SUM, AVG, MIN, MAX	8.11 – 8.15
D	GROUP BY on one column, two columns and an expression	8.16 – 8.20
E	HAVING	8.21 – 8.24
F	UPDATE and DELETE	8.25 – 8.27

A. Selecting Rows with WHERE

8.1 International airports in the network

```sql
SELECT airport_code, airport_name, city, country
FROM airports
WHERE country <> 'India';
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT airport_code, airport_name, city, country
    -> FROM airports
    -> WHERE country <> 'India';
+--------------+----------------------------+-----------+----------------------+
| airport_code | airport_name               | city      | country              |
+--------------+----------------------------+-----------+----------------------+
| DXB          | Dubai International Airport | Dubai     | United Arab Emirates |
| SIN          | Singapore Changi Airport    | Singapore | Singapore            |
+--------------+----------------------------+-----------+----------------------+
2 rows in set (0.00 sec)
```

What it shows: A simple inequality on one column: the destinations that need passports and immigration checks.

8.2 Long routes (over 1,000 km) that take more than two hours

```
SELECT flight_number, origin_code, destination_code, distance_km, duration_min
FROM routes
WHERE distance_km > 1000
  AND duration_min > 120;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_number, origin_code, destination_code, distance_km, duration_min
    -> FROM routes
    -> WHERE distance_km > 1000
    ->   AND duration_min > 120;
+---------------+-------------+------------------+-------------+--------------+
| flight_number | origin_code | destination_code | distance_km | duration_min |
+---------------+-------------+------------------+-------------+--------------+
| SW101         | HYD         | DEL              |        1265 |          130 |
| SW102         | DEL         | HYD              |        1265 |          135 |
| SW801         | HYD         | DXB              |        2600 |          245 |
| SW802         | DXB         | HYD              |        2600 |          230 |
| SW901         | HYD         | SIN              |        3200 |          300 |
+---------------+-------------+------------------+-------------+--------------+
5 rows in set (0.00 sec)
```

What it shows: Both conditions must be true (AND). These are the routes that get hot meals and the larger aircraft.

8.3 Flights departing in the week of 20–26 October (BETWEEN)

```
SELECT flight_id, route_id, departure_time, flight_status
FROM flights
WHERE departure_time BETWEEN '2026-10-20 00:00:00' AND '2026-10-26 23:59:59';
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_id, route_id, departure_time, flight_status
    -> FROM flights
    -> WHERE departure_time BETWEEN '2026-10-20 00:00:00' AND '2026-10-26 23:59:59';
+-----------+----------+---------------------+---------------+
| flight_id | route_id | departure_time      | flight_status |
+-----------+----------+---------------------+---------------+
|         5 |        1 | 2026-10-25 06:10:00 | Arrived       |
|        11 |        2 | 2026-10-26 09:30:00 | Delayed       |
|        16 |        3 | 2026-10-20 07:40:00 | Arrived       |
|        22 |        4 | 2026-10-21 10:20:00 | Arrived       |
|        28 |        5 | 2026-10-22 08:15:00 | Arrived       |
|        34 |        6 | 2026-10-23 11:05:00 | Arrived       |
|        41 |        7 | 2026-10-24 13:30:00 | Arrived       |
|        47 |        8 | 2026-10-25 16:10:00 | Scheduled     |
|        53 |        9 | 2026-10-26 15:45:00 | Scheduled     |
|        58 |       10 | 2026-10-20 12:10:00 | Arrived       |
|        64 |       11 | 2026-10-21 17:20:00 | Arrived       |
|        70 |       12 | 2026-10-22 19:10:00 | Arrived       |
|        85 |       15 | 2026-10-20 23:55:00 | Arrived       |
|        91 |       16 | 2026-10-26 14:20:00 | Scheduled     |
+-----------+----------+---------------------+---------------+
14 rows in set (0.00 sec)
```

What it shows: BETWEEN is inclusive, so the upper bound is the last second of 26 October.

8.4 Passengers whose last name is Reddy or Rao, or who are not Indian (LIKE, IN)

```
SELECT passenger_id, first_name, last_name, nationality
FROM passengers
WHERE last_name IN ('Reddy', 'Rao')
   OR nationality NOT IN ('Indian');
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT passenger_id, first_name, last_name, nationality
    -> FROM passengers
    -> WHERE last_name IN ('Reddy', 'Rao')
    ->    OR nationality NOT IN ('Indian');
+--------------+------------+-------------+-------------+
| passenger_id | first_name | last_name   | nationality |
+--------------+------------+-------------+-------------+
|            1 | Srinivas   | Reddy       | Indian      |
|            2 | Lakshmi    | Reddy       | Indian      |
|            3 | Harika     | Reddy       | Indian      |
|            4 | Varun      | Reddy       | Indian      |
|           18 | Prakash    | Rao         | Indian      |
|           19 | Sunitha    | Rao         | Indian      |
|           20 | Sahithi    | Rao         | Indian      |
|           21 | Rohith     | Rao         | Indian      |
|           42 | James      | Wilson      | British     |
|           43 | Wei Ling   | Tan         | Singaporean |
|           44 | Omar       | Al Mansoori | Emirati     |
|           45 | Sarah      | Thompson    | British     |
+--------------+------------+-------------+-------------+
12 rows in set (0.00 sec)
```

What it shows: IN tests a list of values; OR combines the two groups of passengers.

8.5 Seats by the window: tickets in seat A or F (LIKE patterns)

```sql
SELECT ticket_id, flight_id, passenger_id, seat_number, class_code
FROM ticket_details
WHERE (seat_number LIKE '%A' OR seat_number LIKE '%F')
  AND class_code = 'BU';
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT ticket_id, flight_id, passenger_id, seat_number, class_code
    -> FROM ticket_details
    -> WHERE (seat_number LIKE '%A' OR seat_number LIKE '%F')
    ->   AND class_code = 'BU';
+-----------+-----------+--------------+-------------+------------+
| ticket_id | flight_id | passenger_id | seat_number | class_code |
+-----------+-----------+--------------+-------------+------------+
|         6 |         5 |           13 | 4A          | BU         |
|         8 |         5 |           15 | 6A          | BU         |
|        29 |        42 |           24 | 4F          | BU         |
|        73 |        36 |           34 | 3A          | BU         |
|        89 |        53 |           35 | 5F          | BU         |
|       195 |        24 |           17 | 4F          | BU         |
|       222 |        36 |           37 | 6A          | BU         |
|       261 |        77 |           14 | 4A          | BU         |
|       264 |        89 |           42 | 6F          | BU         |
|       288 |        10 |            8 | 5A          | BU         |
|       289 |        56 |           11 | 4F          | BU         |
+-----------+-----------+--------------+-------------+------------+
11 rows in set (0.00 sec)
```

What it shows: `'%A'` matches any seat number ending in A. The query lists window seats sold in Business class.

8.6 Bookings that are not yet paid

```sql
SELECT booking_id, booking_reference, passenger_id, booking_date
FROM bookings
WHERE status = 'Pending';
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT booking_id, booking_reference, passenger_id, booking_date
    -> FROM bookings
    -> WHERE status = 'Pending';
+------------+-------------------+--------------+---------------------+
| booking_id | booking_reference | passenger_id | booking_date        |
+------------+-------------------+--------------+---------------------+
|         35 | QRJDXX            |           38 | 2026-10-25 07:00:00 |
|         51 | ERF28Y            |            3 | 2026-10-24 18:00:00 |
|        169 | W4R2QT            |           11 | 2026-10-24 21:00:00 |
|        175 | V4WPRF            |            1 | 2026-10-25 04:00:00 |
+------------+-------------------+--------------+---------------------+
4 rows in set (0.00 sec)
```

What it shows: These PNRs hold seats without payment; the airline releases them if payment does not arrive.

8.7 Failed payment attempts above ₹10,000

```sql
SELECT payment_id, booking_id, amount, payment_method, transaction_date
FROM payments
WHERE payment_status = 'Failed'
  AND amount > 10000;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT payment_id, booking_id, amount, payment_method, transaction_date
    -> FROM payments
    -> WHERE payment_status = 'Failed'
    ->   AND amount > 10000;
+------------+------------+----------+----------------+---------------------+
| payment_id | booking_id | amount   | payment_method | transaction_date    |
+------------+------------+----------+----------------+---------------------+
|          3 |          3 | 62592.00 | Debit Card     | 2026-09-08 09:17:00 |
|         52 |         48 | 15520.00 | Debit Card     | 2026-09-25 17:08:00 |
|         55 |         50 | 19740.00 | Debit Card     | 2026-08-21 05:17:00 |
|         91 |         80 | 29680.00 | Credit Card    | 2026-09-15 23:15:00 |
|        121 |        106 | 13650.00 | Credit Card    | 2026-08-23 23:20:00 |
|        128 |        111 | 11160.00 | Credit Card    | 2026-09-21 16:09:00 |
+------------+------------+----------+----------------+---------------------+
6 rows in set (0.00 sec)
```

What it shows: High-value failed attempts, the ones the payments team investigates first.

B. Sorting with ORDER BY

8.8 The ten most expensive tickets sold

```sql
SELECT ticket_id, flight_id, class_code, fare
FROM ticket_details
ORDER BY fare DESC
LIMIT 10;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT ticket_id, flight_id, class_code, fare
    -> FROM ticket_details
    -> ORDER BY fare DESC
    -> LIMIT 10;
+-----------+-----------+------------+----------+
| ticket_id | flight_id | class_code | fare     |
+-----------+-----------+------------+----------+
|        85 |        84 | FC         | 76670.00 |
|       167 |        83 | FC         | 75075.00 |
|       261 |        77 | BU         | 36384.00 |
|        80 |        83 | PE         | 21840.00 |
|         9 |        54 | BU         | 21664.00 |
|         6 |         5 | BU         | 20864.00 |
|         7 |         5 | BU         | 20864.00 |
|         8 |         5 | BU         | 20864.00 |
|        22 |         6 | BU         | 19872.00 |
|       288 |        10 | BU         | 18560.00 |
+-----------+-----------+------------+----------+
10 rows in set (0.00 sec)
```

What it shows: Descending order on fare plus LIMIT gives a top-10 list; it is dominated by Business and First on the long routes.

8.9 Routes ordered by distance, then by flight number

```sql
SELECT flight_number, origin_code, destination_code, distance_km
FROM routes
ORDER BY distance_km DESC, flight_number ASC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_number, origin_code, destination_code, distance_km
    -> FROM routes
    -> ORDER BY distance_km DESC, flight_number ASC;
+---------------+-------------+------------------+-------------+
| flight_number | origin_code | destination_code | distance_km |
+---------------+-------------+------------------+-------------+
| SW901         | HYD         | SIN              |        3200 |
| SW801         | HYD         | DXB              |        2600 |
| SW802         | DXB         | HYD              |        2600 |
| SW101         | HYD         | DEL              |        1265 |
| SW102         | DEL         | HYD              |        1265 |
| SW501         | HYD         | CCU              |        1180 |
| SW651         | HYD         | COK              |         860 |
| SW201         | HYD         | BOM              |         620 |
| SW202         | BOM         | HYD              |         620 |
| SW401         | HYD         | MAA              |         520 |
| SW402         | MAA         | HYD              |         520 |
| SW601         | HYD         | GOI              |         520 |
| SW701         | HYD         | VTZ              |         520 |
| SW702         | VTZ         | HYD              |         520 |
| SW301         | HYD         | BLR              |         500 |
| SW302         | BLR         | HYD              |         500 |
+---------------+-------------+------------------+-------------+
16 rows in set (0.00 sec)
```

What it shows: When two routes have the same distance (outbound and return), the second sort key decides the order.

8.10 Upcoming departures, earliest first

```sql
SELECT flight_id, route_id, departure_time, flight_status
FROM flights
WHERE departure_time > '2026-10-25 14:00:00'
ORDER BY departure_time
LIMIT 8;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_id, route_id, departure_time, flight_status
    -> FROM flights
    -> WHERE departure_time > '2026-10-25 14:00:00'
    -> ORDER BY departure_time
    -> LIMIT 8;
+-----------+----------+---------------------+---------------+
| flight_id | route_id | departure_time      | flight_status |
+-----------+----------+---------------------+---------------+
|        47 |        8 | 2026-10-25 16:10:00 | Scheduled     |
|        11 |        2 | 2026-10-26 09:30:00 | Delayed       |
|        91 |       16 | 2026-10-26 14:20:00 | Scheduled     |
|        53 |        9 | 2026-10-26 15:45:00 | Scheduled     |
|        17 |        3 | 2026-10-27 07:40:00 | Scheduled     |
|        59 |       10 | 2026-10-27 12:10:00 | Scheduled     |
|        76 |       13 | 2026-10-27 20:30:00 | Scheduled     |
|        81 |       14 | 2026-10-28 03:30:00 | Scheduled     |
+-----------+----------+---------------------+---------------+
8 rows in set (0.00 sec)
```

What it shows: The departures board: flights after the current time, in time order.

C. Aggregate Functions

8.11 COUNT – size of the operation

```sql
SELECT COUNT(*)                    AS tickets_issued,
       COUNT(DISTINCT passenger_id)  AS different_travellers,
       COUNT(DISTINCT flight_id)     AS flights_with_passengers,
       COUNT(DISTINCT booking_id)    AS bookings
FROM ticket_details;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT COUNT(*)                    AS tickets_issued,
    ->        COUNT(DISTINCT passenger_id)  AS different_travellers,
    ->        COUNT(DISTINCT flight_id)     AS flights_with_passengers,
    ->        COUNT(DISTINCT booking_id)    AS bookings
    -> FROM ticket_details;
+---------------+---------------------+------------------------+----------+
| tickets_issued | different_travellers | flights_with_passengers | bookings |
+---------------+---------------------+------------------------+----------+
|            310 |                  43 |                     84 |      175 |
+---------------+---------------------+------------------------+----------+
1 row in set (0.00 sec)
```

What it shows: `COUNT(DISTINCT …)` counts each traveller, flight or booking once, however many tickets they have.

8.12 SUM – money collected, refunded and still pending

```sql
SELECT SUM(CASE WHEN payment_status = 'Success'  THEN amount ELSE 0 END) AS collected,
       SUM(CASE WHEN payment_status = 'Refunded' THEN amount ELSE 0 END) AS refunded,
       SUM(CASE WHEN payment_status = 'Pending'  THEN amount ELSE 0 END) AS awaiting
FROM payments;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT SUM(CASE WHEN payment_status = 'Success'  THEN amount ELSE 0 END) AS collected,
    ->        SUM(CASE WHEN payment_status = 'Refunded' THEN amount ELSE 0 END) AS refunded,
    ->        SUM(CASE WHEN payment_status = 'Pending'  THEN amount ELSE 0 END) AS awaiting
    -> FROM payments;
+------------+-----------+----------+
| collected  | refunded  | awaiting |
+------------+-----------+----------+
| 1838457.00 | 167602.00 | 28032.00 |
+------------+-----------+----------+
1 row in set (0.00 sec)
```

What it shows: Conditional sums split one column into three totals. Failed attempts are excluded because no money moved.

8.13 AVG – average fare and average flight length

```sql
SELECT ROUND(AVG(fare), 2) AS avg_fare_all_tickets
FROM ticket_details
WHERE ticket_status <> 'Cancelled';

SELECT ROUND(AVG(duration_min)) AS avg_duration_min, ROUND(AVG(distance_km)) AS avg_distance_km
FROM routes;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT ROUND(AVG(fare), 2) AS avg_fare_all_tickets
    -> FROM ticket_details
    -> WHERE ticket_status <> 'Cancelled';
+----------------------+
| avg_fare_all_tickets |
+----------------------+
|              6642.31 |
+----------------------+
1 row in set (0.00 sec)

mysql> SELECT ROUND(AVG(duration_min)) AS avg_duration_min, ROUND(AVG(distance_km)) AS avg_distance_km
    -> FROM routes;
+------------------+-----------------+
| avg_duration_min | avg_distance_km |
+------------------+-----------------+
|              121 |            1113 |
+------------------+-----------------+
1 row in set (0.00 sec)
```

What it shows: AVG over a filtered set (live tickets only) and over two columns of the routes table.

8.14 MIN and MAX – cheapest and dearest base price, first and last departure

```sql
SELECT MIN(base_price) AS lowest_base, MAX(base_price) AS highest_base,
       MIN(departure_time) AS first_departure, MAX(departure_time) AS last_departure
FROM flights;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT MIN(base_price) AS lowest_base, MAX(base_price) AS highest_base,
    ->        MIN(departure_time) AS first_departure, MAX(departure_time) AS last_departure
    -> FROM flights;
+-------------+--------------+---------------------+---------------------+
| lowest_base | highest_base | first_departure     | last_departure      |
+-------------+--------------+---------------------+---------------------+
|     2910.00 |     14290.00 | 2026-09-26 13:30:00 | 2026-11-06 11:05:00 |
+-------------+--------------+---------------------+---------------------+
1 row in set (0.00 sec)
```

What it shows: MIN/MAX on prices and on DATETIME values in a single query.

8.15 All five aggregates on successful payments

```
SELECT COUNT(*)              AS payments,
       SUM(amount)           AS revenue,
       ROUND(AVG(amount), 2) AS avg_booking_value,
       MIN(amount)           AS smallest,
       MAX(amount)           AS largest
FROM payments
WHERE payment_status = 'Success';
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT COUNT(*)              AS payments,
    ->        SUM(amount)           AS revenue,
    ->        ROUND(AVG(amount), 2) AS avg_booking_value,
    ->        MIN(amount)           AS smallest,
    ->        MAX(amount)           AS largest
    -> FROM payments
    -> WHERE payment_status = 'Success';
+----------+------------+-------------------+----------+----------+
| payments | revenue    | avg_booking_value | smallest | largest  |
+----------+------------+-------------------+----------+----------+
|      155 | 1838457.00 |          11861.01 |  2910.00 | 76670.00 |
+----------+------------+-------------------+----------+----------+
1 row in set (0.00 sec)
```

What it shows: The key revenue figures of the period. WHERE removes failed, pending and refunded attempts before aggregating.

D. Grouping with GROUP BY

8.16 Tickets and revenue by travel class

```
SELECT class_code, COUNT(*) AS tickets, SUM(fare) AS revenue, ROUND(AVG(fare), 2) AS avg_fare
FROM ticket_details
WHERE ticket_status <> 'Cancelled'
GROUP BY class_code
ORDER BY revenue DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT class_code, COUNT(*) AS tickets, SUM(fare) AS revenue, ROUND(AVG(fare), 2) AS avg_fare
    -> FROM ticket_details
    -> WHERE ticket_status <> 'Cancelled'
    -> GROUP BY class_code
    -> ORDER BY revenue DESC;
+------------+---------+------------+----------+
| class_code | tickets | revenue    | avg_fare |
+------------+---------+------------+----------+
| EC         |     241 | 1199000.00 |  4975.10 |
| BU         |      22 |  350848.00 | 15947.64 |
| PE         |      16 |  164896.00 | 10306.00 |
| FC         |       2 |  151745.00 | 75872.50 |
+------------+---------+------------+----------+
4 rows in set (0.00 sec)
```

What it shows: Economy sells the most seats, but each Business ticket earns more than three times as much.

8.17 Flights per operational status

```
SELECT flight_status, COUNT(*) AS flights
FROM flights
GROUP BY flight_status;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_status, COUNT(*) AS flights
    -> FROM flights
    -> GROUP BY flight_status;
+---------------+---------+
| flight_status | flights |
+---------------+---------+
| Arrived       |      61 |
| Scheduled     |      28 |
| Delayed       |       1 |
| Cancelled     |       2 |
+---------------+---------+
4 rows in set (0.00 sec)
```

What it shows: One row per status: the operations summary as of the snapshot time.

8.18 Bookings made per month

```sql
SELECT DATE_FORMAT(booking_date, '%Y-%m') AS month,
       COUNT(*)                           AS bookings,
       SUM(status = 'Cancelled')          AS cancelled
FROM bookings
GROUP BY DATE_FORMAT(booking_date, '%Y-%m')
ORDER BY month;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT DATE_FORMAT(booking_date, '%Y-%m') AS month,
    ->        COUNT(*)                           AS bookings,
    ->        SUM(status = 'Cancelled')          AS cancelled
    -> FROM bookings
    -> GROUP BY DATE_FORMAT(booking_date, '%Y-%m')
    -> ORDER BY month;
+---------+----------+-----------+
| month   | bookings | cancelled |
+---------+----------+-----------+
| 2026-07 |        2 |         0 |
| 2026-08 |       37 |         4 |
| 2026-09 |       89 |         8 |
| 2026-10 |       47 |         4 |
+---------+----------+-----------+
4 rows in set (0.00 sec)
```

What it shows: Grouping on a formatted date shows the booking curve; SUM of a condition counts cancellations per month.

8.19 Payment attempts by method and outcome

```sql
SELECT payment_method, payment_status, COUNT(*) AS attempts
FROM payments
GROUP BY payment_method, payment_status
ORDER BY payment_method, attempts DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT payment_method, payment_status, COUNT(*) AS attempts
    -> FROM payments
    -> GROUP BY payment_method, payment_status
    -> ORDER BY payment_method, attempts DESC;
+----------------+----------------+----------+
| payment_method | payment_status | attempts |
+----------------+----------------+----------+
| Credit Card    | Success        |       42 |
| Credit Card    | Failed         |       13 |
| Credit Card    | Pending        |        3 |
| Credit Card    | Refunded       |        2 |
| Debit Card     | Success        |       27 |
| Debit Card     | Failed         |       11 |
| Debit Card     | Refunded       |        5 |
| Debit Card     | Pending        |        1 |
| Net Banking    | Success        |       24 |
| Net Banking    | Refunded       |        4 |
| UPI            | Success        |       62 |
| UPI            | Refunded       |        5 |
+----------------+----------------+----------+
12 rows in set (0.00 sec)
```

What it shows: Grouping on two columns gives the success and failure count of every payment method.

8.20 Ticket status breakdown

```sql
SELECT ticket_status, COUNT(*) AS tickets
FROM ticket_details
GROUP BY ticket_status
ORDER BY tickets DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT ticket_status, COUNT(*) AS tickets
    -> FROM ticket_details
    -> GROUP BY ticket_status
    -> ORDER BY tickets DESC;
+---------------+---------+
| ticket_status | tickets |
+---------------+---------+
| Boarded       |     179 |
| Booked        |      93 |
| Cancelled     |      29 |
| Checked-in    |       5 |
| No-show       |       4 |
+---------------+---------+
5 rows in set (0.00 sec)
```

What it shows: Boarded tickets are past flights, Booked and Checked-in are future ones; Cancelled includes the two cancelled flights.

E. Filtering Groups with HAVING

8.21 Frequent flyers: travellers with more than six tickets

```sql
SELECT passenger_id, COUNT(*) AS tickets
FROM ticket_details
WHERE ticket_status <> 'Cancelled'
GROUP BY passenger_id
HAVING COUNT(*) > 6
ORDER BY tickets DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT passenger_id, COUNT(*) AS tickets
    -> FROM ticket_details
    -> WHERE ticket_status <> 'Cancelled'
    -> GROUP BY passenger_id
    -> HAVING COUNT(*) > 6
    -> ORDER BY tickets DESC;
+--------------+---------+
| passenger_id | tickets |
+--------------+---------+
|            1 |      18 |
|            2 |      16 |
|            3 |      15 |
|            4 |      14 |
|           14 |      14 |
|           15 |      13 |
|           13 |      11 |
|            8 |       8 |
|           24 |       8 |
|           26 |       8 |
|           10 |       7 |
|           16 |       7 |
|           19 |       7 |
|           20 |       7 |
|           39 |       7 |
|           42 |       7 |
+--------------+---------+
16 rows in set (0.00 sec)
```

What it shows: HAVING filters after grouping, keeping candidates for a loyalty programme.

8.22 Busy flights: more than six live tickets

```sql
SELECT flight_id, COUNT(*) AS seats_sold
FROM ticket_details
WHERE ticket_status <> 'Cancelled'
GROUP BY flight_id
HAVING seats_sold > 6
ORDER BY seats_sold DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_id, COUNT(*) AS seats_sold
    -> FROM ticket_details
    -> WHERE ticket_status <> 'Cancelled'
    -> GROUP BY flight_id
    -> HAVING seats_sold > 6
    -> ORDER BY seats_sold DESC;
+-----------+------------+
| flight_id | seats_sold |
+-----------+------------+
|        36 |         12 |
|         8 |         10 |
|        56 |         10 |
|        37 |          9 |
|        29 |          8 |
|        61 |          8 |
|         6 |          7 |
|        25 |          7 |
|        35 |          7 |
|        74 |          7 |
+-----------+------------+
10 rows in set (0.00 sec)
```

What it shows: MySQL allows the alias `seats_sold` in HAVING.

8.23 Payment methods with a failure rate above 10 %

```
SELECT payment_method,
       COUNT(*)                                          AS attempts,
       SUM(payment_status = 'Failed')                    AS failed,
       ROUND(100 * SUM(payment_status = 'Failed') / COUNT(*), 1) AS failure_pct
FROM payments
GROUP BY payment_method
HAVING failure_pct > 10;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT payment_method,
    ->        COUNT(*)                                          AS attempts,
    ->        SUM(payment_status = 'Failed')                    AS failed,
    ->        ROUND(100 * SUM(payment_status = 'Failed') / COUNT(*), 1) AS failure_pct
    -> FROM payments
    -> GROUP BY payment_method
    -> HAVING failure_pct > 10;
+----------------+----------+--------+-------------+
| payment_method | attempts | failed | failure_pct |
+----------------+----------+--------+-------------+
| Debit Card     |       44 |     11 |        25.0 |
| Credit Card    |       60 |     13 |        21.7 |
+----------------+----------+--------+-------------+
2 rows in set (0.00 sec)
```

What it shows: The condition uses a percentage computed from two aggregates, which is only possible in HAVING.

8.24 Bookings worth more than ₹20,000

```
SELECT booking_id, COUNT(*) AS tickets, SUM(fare) AS booking_value
FROM ticket_details
WHERE ticket_status <> 'Cancelled'
GROUP BY booking_id
HAVING SUM(fare) > 20000
ORDER BY booking_value DESC;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT booking_id, COUNT(*) AS tickets, SUM(fare) AS booking_value
    -> FROM ticket_details
    -> WHERE ticket_status <> 'Cancelled'
    -> GROUP BY booking_id
    -> HAVING SUM(fare) > 20000
    -> ORDER BY booking_value DESC;
+------------+---------+---------------+
| booking_id | tickets | booking_value |
+------------+---------+---------------+
|         45 |       1 |      76670.00 |
|         87 |       1 |      75075.00 |
|          3 |       3 |      62592.00 |
|         36 |       4 |      46120.00 |
|         27 |       4 |      40040.00 |
|        144 |       1 |      36384.00 |
|         34 |       4 |      33408.00 |
|        157 |       3 |      30780.00 |
|         58 |       2 |      30592.00 |
|        123 |       3 |      30540.00 |
|         80 |       8 |      29680.00 |
|        170 |       8 |      28360.00 |
|          6 |       8 |      27880.00 |
|        103 |       2 |      26944.00 |
|          2 |       4 |      24840.00 |
|         39 |       4 |      24720.00 |
|        164 |       2 |      23808.00 |
|         42 |       1 |      21840.00 |
|          4 |       1 |      21664.00 |
|         37 |       2 |      21520.00 |
|        140 |       2 |      21520.00 |
|        162 |       2 |      21520.00 |
|         26 |       4 |      21000.00 |
|         49 |       2 |      20590.00 |
+------------+---------+---------------+
24 rows in set (0.00 sec)
```

What it shows: High-value bookings are typically families or business-class travellers.

F. Changing Data

8.25 UPDATE – a larger aircraft is assigned to a flight

```sql
SELECT flight_id, flight_number, capacity, seats_sold, load_factor_pct
FROM v_flight_occupancy WHERE flight_id = 17;

UPDATE flights SET aircraft_id = 3 WHERE flight_id = 17;

SELECT flight_id, flight_number, capacity, seats_sold, load_factor_pct
FROM v_flight_occupancy WHERE flight_id = 17;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT flight_id, flight_number, capacity, seats_sold, load_factor_pct
    -> FROM v_flight_occupancy WHERE flight_id = 17;
+-----------+---------------+----------+------------+-----------------+
| flight_id | flight_number | capacity | seats_sold | load_factor_pct |
+-----------+---------------+----------+------------+-----------------+
|        17 | SW201         |      180 |          5 |             2.8 |
+-----------+---------------+----------+------------+-----------------+
1 row in set (0.00 sec)

mysql> UPDATE flights SET aircraft_id = 3 WHERE flight_id = 17;
Query OK, 1 row affected (0.05 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT flight_id, flight_number, capacity, seats_sold, load_factor_pct
    -> FROM v_flight_occupancy WHERE flight_id = 17;
+-----------+---------------+----------+------------+-----------------+
| flight_id | flight_number | capacity | seats_sold | load_factor_pct |
+-----------+---------------+----------+------------+-----------------+
|        17 | SW201         |      216 |          5 |             2.3 |
+-----------+---------------+----------+------------+-----------------+
1 row in set (0.00 sec)
```

What it shows: Because capacity belongs to the aircraft (Assignment 3), changing one foreign key updates the capacity everywhere. No seat count has to be edited by hand.

8.26 UPDATE – a pending payment succeeds and the booking is confirmed

```sql
UPDATE payments SET payment_status = 'Success' WHERE payment_id = 57;
UPDATE bookings SET status = 'Confirmed' WHERE booking_id = 51;

SELECT booking_id, booking_reference, status, total_fare, amount_paid
FROM v_booking_totals WHERE booking_id = 51;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> UPDATE payments SET payment_status = 'Success' WHERE payment_id = 57;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> UPDATE bookings SET status = 'Confirmed' WHERE booking_id = 51;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT booking_id, booking_reference, status, total_fare, amount_paid
    -> FROM v_booking_totals WHERE booking_id = 51;
+------------+-------------------+-----------+------------+-------------+
| booking_id | booking_reference | status    | total_fare | amount_paid |
+------------+-------------------+-----------+------------+-------------+
|         51 | ERF28Y            | Confirmed |   14120.00 |    14120.00 |
+------------+-------------------+-----------+------------+-------------+
1 row in set (0.00 sec)
```

What it shows: Two updates move booking ERF28Y from Pending to Confirmed. The view confirms that the amount paid equals the total fare.

8.27 DELETE – housekeeping of old failed payment attempts

```sql
SELECT COUNT(*) AS old_failed FROM payments
WHERE payment_status = 'Failed' AND transaction_date < '2026-09-01';

DELETE FROM payments
WHERE payment_status = 'Failed'
  AND transaction_date < '2026-09-01';

SELECT payment_status, COUNT(*) AS attempts FROM payments GROUP BY payment_status;
```

MySQL 8.0 Command Line Client — airline_reservation

```
mysql> SELECT COUNT(*) AS old_failed FROM payments
    -> WHERE payment_status = 'Failed' AND transaction_date < '2026-09-01';
+------------+
| old_failed |
+------------+
|          3 |
+------------+
1 row in set (0.00 sec)

mysql> DELETE FROM payments
    -> WHERE payment_status = 'Failed'
    ->   AND transaction_date < '2026-09-01';
Query OK, 3 rows affected (0.00 sec)

mysql> SELECT payment_status, COUNT(*) AS attempts FROM payments GROUP BY payment_status;
+----------------+----------+
| payment_status | attempts |
+----------------+----------+
| Success        |      156 |
| Failed         |       21 |
| Refunded       |       16 |
| Pending        |        3 |
+----------------+----------+
4 rows in set (0.00 sec)
```

What it shows: The WHERE clause limits the delete to failed attempts older than 1 September. Successful payments and recent failures are untouched.

Conclusion

Single-table queries already answer many of an airline's daily questions. WHERE finds unpaid bookings and failed payments, ORDER BY produces departure boards and top-fare lists, and the aggregate functions measure revenue and volume. GROUP BY breaks the figures down by class, status, month and payment method, and HAVING keeps only frequent flyers, busy flights or risky payment methods. The DML statements show that the normalized design makes changes simple and safe: swapping an aircraft updates capacity through a single foreign key.